

Data Structures

In order to complete this project we used a variety of typedef data structures. We made an enumeration for all the recipe types for ease of use later. We created structs for the following: pantry, fridge, kitchen, recipeCounter, baker resources, and bakerData. We also created an all text colors array that held all the possible text output color options.

There were some structs made to represent the recipe requirements for each of the recipes. This was from a previous idea that was scrapped. Additionally, all of the variables and data structure definitions discussed in this paragraph are global variables.

Sem_wait & Sem_signal function

This function was made for ease of use when it came to having to decrement the semaphore value by 1, this way it would more closely relate to the general discussion we had in our slides and we could abstract away the exact steps needed each time, especially since we will be calling this code multiple times. This function takes 2 parameter, the first is semid which is the actual semaphore we want to perform an operation on, and the second is semaphore set number which we want to do this operation on.

Print_baker_state function

This function is to print the data about the bakers to help us ensure that the program is running smoothly. Plus, it's a nice way to ensure all the baker information is tracked properly

Main Function

It asks for the number of bakers first and based on this number allocates the necessary amount of space. Then the main function creates the necessary semaphores and sets their values: pantry, fridge, kitchen, recipe_counter. Then it creates shared memory for the necessary regions: pantry, fridge, kitchen, and recipe_counter. Then after this, there's some setup that needs to be done before we can call the multi-threaded function.

Semaphores

Implemented using System V semaphores to enforce mutual exclusion and resource availability across the pantry, fridge, kitchen, and recipe counter. The *sem_wait()* function decrements a semaphore to lock a resource, while *sem_signal()* increments it to release the resource, ensuring thread-safe operations. For example, the

pantry's *pantry_semid* is initialized to 1 to allow single-baker access, while the kitchen's *kitchen_semid* is set to 3, 5, and 2 for bowls, spoons, and mixers, respectively. This semaphore-based synchronization is critical to preventing race conditions and deadlocks in the concurrent environment.

Recipes

Each baker is required to complete five recipes: cookies, pancakes, pizza dough, soft pretzels, and cinnamon rolls. Defined using an enumeration structure, *Recipe_type*, these recipes specify unique ingredient combinations sourced from the pantry or fridge. The recipes are assigned to bakers via a semaphore and shared memory region. A dedicated semaphore *recipe_semid* manages the recipe process. Bakers are assigned to a random recipe. See below for more details.

Recipe Counter

This manages the assignment of recipes, implemented as a shared memory structure containing an integer *next_recipe*. This counter cycles through the five recipes, incremented after each assignment to ensure an even distribution of tasks among bakers. This way there's the highest chance of a potential deadlock occurring, and we can ensure we have prevented it. Protected by the *recipe_semid* semaphore, the counter guarantees that only one baker accesses it at a time, maintaining order in the recipe selection process. This mechanism simulates a coordinated kitchen where bakers systematically progress through recipes. However, the initial value of this recipe counter is randomly assigned. Thus, providing random recipe assignment.

Baker Resources

This tracks each baker's current holdings in a *BakerResources* structure, including integers for ingredients and boolean flags for utensils. This structure is dynamically updated as bakers collect and release resources, resetting after each recipe completion. It ensures state management, allowing the program to verify that a baker has all necessary components before mixing and baking. Before the multi-threaded function is ran, we initialize all the baker data and resources accordingly.

We then create the appropriate amount of threads and call our *baker_function* function and pass to it the baker data.

Once all the bakers complete all recipes and all the threads are joined back to the main thread, we print the final state of the kitchen resources, and then cleanup all of our semaphores and shared memory and free the allocated space.

Multi-Threaded baker_Function

This function controls the baking process: selecting a recipe, collecting ingredients from the pantry and fridge, acquiring kitchen utensils, mixing ingredients, and baking. Each baker has a unique ID and a color for output. The baker function handles resource issues and management, and also has interactions with shared resources. Bakers complete five recipes, cycling through the recipe sequence, with their progress tracked via a *numRecipesCompleted* counter.

We set up all the data we need for this function, creating our variables and assigning them the matching data passed in. We make sure to typecast all of our necessary data passed, like the *recipe_type* for instance.

Now, depending on the recipe, different items need to be acquired. So, to handle this is a switch statement within the while loop. For acquisition of these items, see below:

Pantry

A shared memory segment that stores six ingredients: flour, sugar, yeast, baking soda, salt, and cinnamon. A Pantry structure with integer values tracking availability. A semaphore *pantry_semid* restricts access to one baker at a time, using *sem_wait()* to lock the pantry and *sem_signal()* to release it. When a baker enters the pantry, they decrement the desired ingredient's count (e.g., *pantry_ptr->flour--*) and increment their personal BakerResources (e.g., *resources->flour++*). The project assumes unlimited ingredient availability so this isn't done for all resources, but that's the main idea for why stuff is decremented. Regardless, the semaphore ensures orderly access, mimicking a real kitchen's resource constraints.

Fridge

a shared memory segment with three ingredients—eggs, milk, and butter—defined in a Fridge structure with integer counts. Protected by a semaphore (*fridge_semid*), the fridge allows only one baker to access it at a time via *sem_wait()* and *sem_signal()*. Bakers retrieve ingredients by incrementing their *BakerResources* (e.g., *resources->eggs++*), with the unlimited supply assumption ensuring availability. This synchronized access prevents resource conflicts, maintaining thread safety in the simulation.

Kitchen

This provides tools to prepare ingredients to make the recipes, implemented as a shared memory segment with three limited resources: 3 bowls, 5 spoons, and 2 mixers,

tracked in a *Kitchen* structure. A semaphore (*kitchen_semid*) manages access, with distinct indices for the semaphore sets— for bowls (0), spoons (1), and mixers (2). Bakers acquire one of each tool using *sem_wait()* and release them with *sem_signal()* after mixing.

Once all the ingredients have been acquired and a spoon, bowl, and mixer have been acquired, the ingredients get mixed together and then cooked in the oven, which is a simple *sleep(2)* command. Then once they finish their recipe, a print statement is executed to say so, and then they get assigned the next recipe to cook using the enumeration data structure. After a recipe is assigned, the chef then releases all the stuff they're holding onto and all of their ingredients as well before the next loop's iteration.

At the very end, once the chef has cooked all the recipes, they free their thread-specific memory and space that has been allocated.

Ramsied Event

Per the assignment description: "Assign one of the bakers to have a chance to be 'Ramsied', this means they must release all semaphores and start their current recipe from the beginning. This must be programmed to occur every time the program is run."

We assigned this chance to baker 0. Since baker 0's recipe will be randomly chosen, they will cook the cookie recipe randomly. So, since it has to occur every time the program is run, we just had the "chance" be $\frac{1}{5}$, where the 5 are all 5 potential recipes that the baker could be cooking at any given time. So, whenever baker 0 ends up cooking cookies for the first time (since it will be random), it will cause this chef to be Ramsied. Meaning, if Baker 0 is preparing cookies and hasn't been "Ramsied" before, which is tracked by *has_been_ramsied* property, they must release all ingredients and tools, resetting their *BakerResources* and restarting the recipe. So, it is guaranteed to occur every time the program is run per the description.